

Gamache Avionics and Integration Systems

Interface Control Document for the Pathfinder Avionics System

DOCUMENT NUMBER	RELEASE/REVISION	RELEASE DATE
747871BLEP-01	**NEW**	**30/10/2024**

CONTENT OWNER:
Gamache Avionics and Integration Systems (GAINS)

Copyright © 2024 Jordan Gamache. All rights reserved.

1. Introduction..... 2

2. Glossary..... 3

3. References..... 4

4. Software..... 5

4.1. Software Development..... 5

4.1.1. IDE..... 5

4.2. Operational Requirements..... 6

5. Hardware..... 7

5.1. Avionics Hardware..... 7

5.1.1. Adafruit Feather..... 7

5.1.1.1. Main CPU - Telemetry..... 8

5.1.1.2. Secondary CPU - I/O..... 8

5.1.2. Sensors..... 8

6. Rotation Control..... 10

6.1. Kalman Filter..... 10

6.2. Proportional Integral Derivative Controller..... 10

7. Payload Data Transmit Protocol..... 12

1. Introduction

The purpose of this document is to outline the specifications of the design and function(s) of the BLEP Avionics system for the Pathfinder rocket program. BLEP is an integrated avionics system that is capable of performing a series of tasks including recording and assimilating sensor data, calculating state values and movement parameters, identifying and correcting moments of rotation during ascent, communicating over various network protocols with a “ground station”, and ejecting a parachute for safely returning the vehicle to the ground.

This document seeks to describe different aspects of the vehicle, including the software design requirements, hardware schematics, rotation control logic, and data payload communication protocols. This document does not cover any “under-the-hood” practices, such as how the flight software code is built and compiled, nor does it go into detail about the airframe construction, as the scope of this document is not intended to cover the vehicle as a whole, rather it focuses on the avionics systems’ design and operation.

The purpose of designing an avionics system for a model rocket is to be able to remotely control event stages, record and analyze flight data, transmit data in flight to remotely monitor the vehicle state, and to

2. Glossary

ARINC	Aeronautical Radio, Incorporated
BLEP	Big Launch Elevating the Payload
CFD	Computational Fluid Dynamics
COTS	Commercial Off The Shelf
CPU	Central Processing Unit
CRC	Cyclic Redundancy Check
DOF	Degrees Of Freedom
FCSN	Frame Check Sequence Number
FS	Functional Status
FSW	Flight Software
IDE	Integrated Development Environment
IMU	Inertial Measurement Unit
MPU	Main Processing Unit
PBIT	Power on Built-In Test
PCB	Printed Circuit Board
PDTP	Payload Data Transmit Protocol
RAM	Random Access Memory
RCS	Rotation Control System (a.k.a Reaction Control System)
SPU	Secondary Processing Unit
SRAM	Static Random Access Memory
TSU	Temperature Sensing Unit

3. References

"Quaternions and Spatial Rotation." *Wikipedia*, Wikimedia Foundation, 19 Oct. 2024, en.wikipedia.org/wiki/Quaternions_and_spatial_rotation.

"Kalman Filter." *Wikipedia*, Wikimedia Foundation, 23 Oct. 2024, en.wikipedia.org/wiki/Kalman_filter.

"Proportional–Integral–Derivative Controller." *Wikipedia*, Wikimedia Foundation, 22 Oct. 2024, en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller.

"Adafruit Feather M0 with RFM95 LoRa Radio - 900MHGz - RadioFruit." *Adafruit,* Limor Fried, 08 Jun. 2016, https://learn.adafruit.com/adafruit-feather-m0-radio-with-lora-radio-module/overview.

"Adafruit Feather 32u4 Adalogger." *Adafruit,* Limor Fried, 11 Nov. 2015, https://learn.adafruit.com/adafruit-feather-32u4-adalogger.

"Adafruit BME280 Humidity + Barometric Pressure + Temperature Sensor Breakout." *Adafruit,* Limor Fried, 24 Jul. 2015, https://learn.adafruit.com/adafruit-bme280-humidity-barometric-pressure-temperature-sensor-breakout.

"MPU6050 6-DoF Accelerometer and Gyro." *Adafruit,* Bryan Siepert, 06 Nov. 2019, https://learn.adafruit.com/mpu6050-6-dof-accelerometer-and-gyro.

4. Software

The avionics flight software is developed on the Arduino platform, which utilizes a modified flavor of the C++ programming language, with some additional compiler tools that make programming a bit easier. For example, in C++, one cannot call a function prior to defining said function, however, in Arduino, functions can be defined anywhere within the file and the code can still be compiled and executed correctly. The Arduino platform was selected for this avionics system for two reasons: (1) the compatibility of Arduino-compiled code with the microcontroller chosen for the system, and (2) C++ is an easy to understand, highly capable, language; considering that C++ is a *compiled* language, which means that the code is translated directly into machine code that can be executed by the computer's processor, it is incredibly fast and efficient.

4.1. Software Development

Configuration

As stated in the section above, the avionics flight software is written in C++, and consists of a few support files that provide a method for easy flight configuration without having to make tedious changes to the core flight computer code. Some variables that are reused frequently were removed from the core code and placed into a *variables.h* file, allowing these values to be set statically, while also preventing any unwanted modification of these variables by using the *#define* keyword. Additionally, some features within the execution flow may need to intermittently be deactivated for testing purposes, thus creating the need for a method to easily modify the flight configuration and flight plan, which is the purpose of the *flightConfiguration.h* input file. This file allows engineers to toggle certain features like verbose logging of data for easier debugging, or a ‘mute’ option which disables all use of the piezo buzzer during operation.

4.1.1. IDE

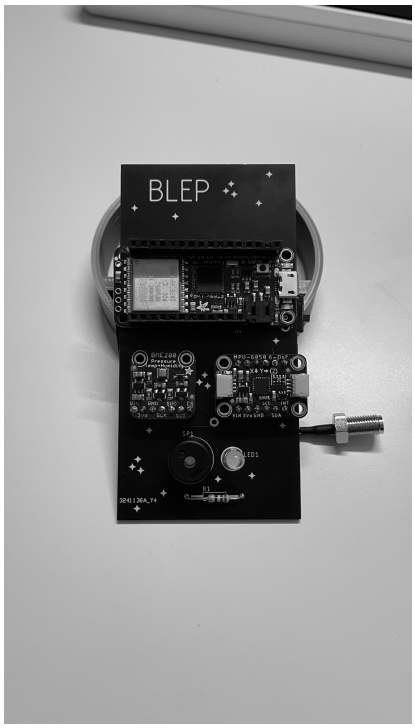
The software was primarily built in the Arduino IDE, due to the easy to use compiler and flashing tools, which improve the efficiency of uploading new code to the avionics system. Visual Studio Code was also used for large portions of refactoring and patches to the software, especially when directly data loading the avionics board was not necessarily required.

4.2. Operational Requirements

FSW-1	The FSW shall provide error handling and must abort any prior commands in the event of an error prior to launch.
FSW-2	The FSW shall log all data to a removable drive to mitigate the risk of telemetry failure so that post-flight analysis can occur.
FSW-3	The FSW shall provide a method for pre-flight configuration and modification of crucial variables (such as SEALEVELPRESSURE).
FSW-4	The FSW shall convert *all* data in accordance with Section 7 [PDTP], regardless of destination.
FSW-5	The FSW shall provide the means to manually override key events during the flight plan. The FSW must be able to, after verifying the correct command was received, perform a manual event trigger such as igniting the engine or deploying the parachutes.
FSW-6	The FSW must be able to encode and decode a CRC checksum hashes to maintain data integrity for all data (logged and transmitted).
FSW-7	The FSW shall provide periodic status updates through network packets and audio/visual cues before, during, and after liftoff has occurred.
FSW-8	The FSW shall be able to detect when the engine has run out of fuel (e.g. thrust = 0) and when descent has begun, and must be able to independently deploy the parachutes, regardless of telemetry state.
FSW-9	The FSW shall log a system error if either the TSU or IMU go offline at any point during operation. The FSW shall reset the RCS control surface back to its idle state to prevent uncontrolled rotation during flight and continue to log all available data points.
FSW-9.1	During a sensor downtime event, the FSW shall make no more than 3 attempts to “restart” the sensor by performing a power-cycle and/or by re-initializing the sensor programmatically. This effort ***must*** not interfere with continuous flight programming or datalogging.
FSW-10	The FSW must be able to automatically switch between modes of telemetry uplink (e.g. RF to WiFi, or vice versa) based on flight program events.
FSW-11	The FSW must continuously write data to the on-board SD Card and implement a form of data integrity/protection to protect against the corruption of data on power off.

5. Hardware

The hardware designed for this vehicle integrates COTS components into a custom-built PCB which is used in conjunction with a series of other COTS components, bridging the gap between a fully custom computer and a simple breadboard with an arduino and sensors. The specific components used were selected for their compatibility as well as their physical attributes, opting for high-performing, low weight boards, where possible. The core flight computer PCB was specifically designed to allow COTS components to be mounted and easily removed in the event that component repair or replacement is needed, or to replicate for future projects.



5.1. Avionics Hardware

5.1.1. Adafruit Feather

The core flight computer is powered by an Adafruit Feather M0 with RFM95 LoRa Radio, which is an M0 microcontroller with an integrated RFM95 “Long Range Wide Area Network” (LoRaWAN) radio chip. The “Feather” is an Adafruit standard microcontroller main board with consistent dimensions that allows for increased modularity and customization through the support of various add-ons in a small form factor. Multiple Feather-compatible boards and “Feather Wing” addons are used throughout the avionics system, as this board format is designed to be highly modular and allows for “stacking” multiple add-on boards for improved flexibility and performance.

5.1.1.1. Main CPU - Telemetry

The Adafruit Feather M0 sports an ATSAMD21G18 ARM Cortex M0 processor, which is a low-power, high-performance microchip with 256 KB of flash memory and 32 KB of SRAM. The M0 (as it will be called going forward) has six serial communication modules and built-in USB device support, and a 1.62V to 3.63V power supply. This chip has 8x more storage and up to 16x more RAM than the Atmega32u4 that was being used in the original design. These changes allow the flight computer core programming file to be much larger while also improving performance with the additional SRAM.

5.1.1.2. Secondary CPU - I/O

There is an additional CPU that provides additional support for certain input and output (I/O) commands. The main CPU (MPU) is certainly capable of performing and processing the I/O signals, but due to the limited size of the Feather boards, there are only a set number of GPIO pins available. It is significantly more efficient to perform the necessary calculations and create the telemetry packets (as defined in Section 7) and send a duplicate packet over I2C or Serial (wire) signal to another CPU to perform any I/O commands. The secondary CPU is also a Feather board, but it utilizes a lower performing processor, the Atmega32u4 referenced in Section 5.1.1.1. This particular board is called the “Adalogger”, as it includes an SD card port soldered directly to the PCB, which this secondary CPU (SPU) will use to log all data packets sent from the MPU and store them on the onboard SD card. In the event that the RF telemetry fails during flight, the MPU will continue to send data packets over serial connection to the SPU to continue logging flight data for analysis post flight.

5.1.2. Sensors

The flight computer utilizes two critical sensors to create a general space state of the vehicle. The BME280 Temperature Sensing Unit (TSU) and MPU6050 6-DOF (also known as the Inertial Measurement Unit (IMU)) sensor are both used in parallel to determine things like vehicle orientation, altitude, rotation, acceleration, temperature, and pressure. The discrete values are read from both sensors and stored within the code as local variables to perform any necessary calculations or process any algorithms to improve data accuracy before sending data through the PDTP. Both sensors implement a PBIT and will throw a system error if they fail this test.

Additionally, if, at any time during the flight, either sensor stops responding, an error status will be recorded, and the computer will attempt to restart the sensor. If the IMU, specifically, fails during the flight, the RCS features will be disabled and the control surface will be locked into its “idle” position to prevent uncontrollable roll.

6. Rotation Control

The avionics system calculates many values during operation, including the assimilation of X, Y, and Z values from the IMU. One of the primary functions of the flight computer is to take these values and calculate spatial rotation through the use of quaternions. It then takes these spatial rotation values, and inversely calculates the difference in rotation required to correct for this moment of rotation. These IMU values are captured at a baud rate of 115200, which is 115,200 bits per second. In order to not overwhelm the MPU with these rapid calculations, a few control theory mechanisms are implemented.

6.1. Kalman Filter

Kalman filtering is a statistics and control theory algorithm that "uses a series of measurements observed over time, including statistical noise and other inaccuracies, to produce estimates of unknown variables that tend to be more accurate than those based on a single measurement, by estimating a joint probability distribution over the variables for each time-step." In practice, this is a method that is used to take in a large amount of data and use statistics to track an "estimate of state" which can be used to significantly reduce "noise" within data. The BLEP flight computer implements Kalman filtering through an Arduino code library that provides the mathematical tools to pass in values to the Kalman filter and receive a highly accurate estimate of the value over time.

6.2. Proportional Integral Derivative Controller

This value is then passed into a Proportional Integral Derivative Controller, or PID controller, which implements a feedback-based control loop to remove error from values in a system. More specifically: "The PID controller automatically compares the desired target value (setpoint or SP) with the actual value of the system (process variable or PV). The difference between these two values is called the error value, denoted as *e(t)*. It then applies corrective actions automatically to bring the PV to the same value as the SP using three methods: The proportional (P) component responds to the current error value by producing an output that is directly proportional to the magnitude of the error. This provides immediate correction based on how far the system is from the desired setpoint. The integral (I) component, in turn, considers the cumulative sum of past errors to address any residual steady-state errors that persist over time, eliminating lingering discrepancies. Lastly, the derivative (D) component predicts future error by assessing the rate of change of the error, which helps to mitigate overshoot and enhance system stability, particularly when the system undergoes rapid changes. The PID controller reduces the likelihood of human error and improves automation". By passing the Kalman filtered values into a PID controller, the values that are returned by the process have been filtered for "statistical noise" and then incorporated into a *feedback-based control loop mechanism* which reduces

the amount of calculations that the flight computer would have to perform each iteration. In practice, this prevents the servos from attempting to rapidly move the RCS control surface every time new data is recorded.

7. Payload Data Transmit Protocol

Following a similar approach to the industry-standard ARINC 664 data format type, this protocol will be less than or equal to 64 bytes and contain a Functional Status Byte, Destination Address, Source Address, Payload, Sequence Number, and a Frame Check Sequence Number (in this case, a CRC-16 checksum value).

FS Byte	**Destination Address**	**Source Address**	**Payload**	**Sequence Number**	**FCSN**
1 byte	2 bytes	2 bytes	40 bytes (minimum)	1 byte	2 bytes

Functional Status Byte
The functional status byte will define the message type as either ‘INFO’, ‘DATA’, ‘ERROR’, or ‘SYNC’.

0 - ***INFO***: Messages under this classification will provide contextual data regarding some behavior or state of the system, that doesn’t contain any data that is required to be parsed, calculated, or logged. An example of this kind of information would be console logging output from the [PBIT].

1 - ***DATA***: Messages under this classification are directly related to avionics reporting and are the “highest” priority data. Type 1 messages contain sensitive payloads required for calculating vehicle state, performance, and other values. Transfer of Type 1 data supports a valid “telemetry” system, allowing a remote machine to receive avionics data payloads. An example of this would be in-flight transmission of yaw, pitch, and roll values for the [RCS].

2 - ***ERROR***: This is the second most “important” Type of message, as any time prior to engine ignition, the system *must* be configured in such a manner that any Type 2 error completely cancels the current task, sets the vehicle into a standby mode that requires user input or a manual reset before proceeding with a launch. Failure to properly configure the system in this way may result in catastrophic conditions before, during, and after liftoff.

3 - ***SYNC***: This data type will simply provide a constant “ping” to confirm telemetry uplink communication is still connected. The payload for a Type 3 message can realistically be anything, as the data will be discarded entirely unless the “DEBUG” flag is set to ***true***.

Destination Address
This is a simple field to designate the flow of network traffic. Values of **130** indicate that the intended destination of the data packet is the ground station, and values of **115** indicate that the intended destination of the data packet is the vehicle itself.

Source Address
This is a simple field to designate the flow of network traffic. Values of **130** indicate that the source of the data packet was the ground station, and values of **115** indicate that the source of the data packet was the vehicle itself.

Payload
The data packet as a whole must be 64 bytes or less, so the payload data itself must be 54 bytes or less in order to accommodate for the additional parameters already allocated within the packet. The format for the payload is as follows:

Byte ID(s)	Description	Bytes (Maximum)
0-4	Temperature	4
4-8	Pressure	4
8-12	Altitude	4
12-16	Humidity	4
16-20	Yaw	4
20-24	Pitch	4
24-28	Roll	4
28-32	Accel_X	4
32-36	Accel_Y	4
36-40	Accel_Z	4
40-54	Pad (optional)	As needed

Note: 4 bytes = 32 bits; 8 bytes = 64 bits

Each data point will be converted into IEEE-754 Single-Precision Binary as described in the table above and sorted within the data message “word” in the above order.

Sequence Number
The payload sequence number is a sequential number ranging from 0-255, and is entirely for data integrity analysis purposes. The sequence number will reset after reaching its maximum of 255, and a sequence reset will be logged.

Frame Check Sequence Number
The Frame Check Sequence Number is a data integrity/validity check that will be calculated on the ground and vehicle sides in order to ensure that the data received was not corrupted during transmission. The use of a CRC-16 checksum will be implemented on the payload data only for simplicity. If increased data integrity checking is required in the distant future, entire packet checksums can be generated, but at this time that is not required.